Data Governance Copilot

Interview Question Bank

Topic 14 — Testing Strategy

12 questions · 4 sections · 240 Tests · 84% Coverage · patch.object · Module Imports · Smoke Tests · fakeredis · Test Pollution

ndational Core concepts | Intermediate Design decisions | Advanced Deep internals | Gotcha Real bugs / traps

---

Test Architecture & Coverage Strategy

Q01 Foundational

Walk through the test suite structure. How are the 240 tests organised across files and what is the strategy behind the naming convention?

Hint: Each test file has a deliberate purpose — explain the progression from day files to specialised files.

Key points to cover:

- Day-numbered files (test_day1.py through test_day20_*.py): track incremental feature delivery — each day's file tests that day's additions; regression safety net as project grew
- test_day14.py: LiteLLM + Redis cache; test_day15.py: retry + HITL; test_day16.py: FastAPI endpoints; test_day17.py: Teams bot + HMAC; test_day18.py: pgvector + MCP client
- test_day19.py: ingestion pipeline (patch.object for psycopg2); test_day20_smoke.py: 15 end-to-end smoke tests; test_day20_coverage.py: coverage boosters; test_day20_production.py: FastAPI + error paths
- Agent-specific files (test_information_agent.py etc.): injection pattern tests — each agent tested with mock service, asserting correct delegation
- test_services.py: protocol conformance — asserts MockXxxService implements IXxxService protocol; factory switching tests
- test_mcp_server.py: MCP tool handler tests — direct handler invocation, no transport
- test_rule_agent.py: CRUD + evaluate invariant — rule registry operations with in-memory store
- Strategy: day files ensure no regression as features layer; specialised files ensure correctness of individual contracts; smoke tests ensure system integration

---

Q02 Foundational

Your project enforces coverage fail_under=80 in pyproject.toml. You achieved 84%. What does this mean and which parts of the codebase are hardest to cover and why?

Hint: 84% is not 100% — understand what the remaining 16% represents.

Key points to cover:

- fail_under=80: pytest --cov fails the CI build if coverage drops below 80% — enforces a minimum standard on every PR
- 84% means 16% of executable lines are not exercised by any test — typically error paths, fallback branches, and external-dependency-heavy code
- Hardest to cover: network-dependent code paths — DatabricksService.query_metrics() real implementation requires a live SQL warehouse; only MockDatabricksService is tested
- Hardest to cover: exception recovery paths — e.g. the 'except ImportError: pass' in checkpointer.py for missing langgraph_checkpoint_sqlite; hard to trigger without uninstalling the package
- Hardest to cover: Streamlit UI (src/ui/app.py) — Streamlit's execution model makes unit testing difficult; typically excluded from coverage measurement
- Hardest to cover: Teams bot HMAC edge cases — testing with real Teams payloads requires a running Teams tenant
- test_day20_coverage.py exists specifically to push coverage from ~78% to 84% — targeted coverage boosters for guardrails, cache, retry, and node edge cases
- 100% is not the goal: 80% is the pragmatic threshold; the remaining 16% is mostly integration-only code that's covered by smoke tests in CI with ENABLE_MOCK=true

---

Q03 Intermediate

Explain the pytest marks used in the project — 'integration' and 'smoke'. How are they configured in pyproject.toml and how does CI use them selectively?

Hint: Custom marks enable selective test execution — explain the CI strategy.

Key points to cover:

- pyproject.toml: [tool.pytest.ini_options] markers = ['integration: requires live services', 'smoke: end-to-end system tests']
- @pytest.mark.integration: tests that require real external services (Databricks, Neon, Redis) — NOT run in standard CI
- @pytest.mark.smoke: end-to-end graph invocation tests with ENABLE_MOCK=true — run in CI but slower than unit tests
- CI standard run: pytest tests/ --cov=src --cov-fail-under=80 -m 'not integration' — excludes integration tests; runs all unit + smoke
- Integration test run: triggered manually or on release branch — requires real SERVICE_URL env vars; runs against staging environment
- Benefit: CI runs in < 2 minutes (no network calls, all mocked); integration tests run in ~10 minutes (real services); developers get fast feedback
- filterwarnings in pyproject.toml: suppresses known deprecation warnings from LangChain and Pydantic V2 migration — keeps CI output clean
- conftest.py shared fixtures: ENABLE_MOCK=true, REDIS_ENABLED=false set via monkeypatch.setenv() — ensures every test runs in mock mode without env var pollution

```
# pyproject.toml [tool.pytest.ini_options] markers = [ 'integration: requires live external services', 'smoke: end-to-end graph invocation with mocks', ] addopts = '--tb=short -q' # CI command (no integration tests) pytest tests/ --cov=src --cov-fail-under=80 -m 'not integration' # Manual integration run pytest tests/ -m integration --env=staging
```

patch.object & Module-Level Import Discipline

Q04 Intermediate

Explain the fundamental rule: 'patch where it is used, not where it is defined.' Give a concrete example from your ingestion pipeline tests.

Hint: This is the most common Python mocking mistake — explain it precisely.

Key points to cover:

- Python's import system: when 'import psycopg2' runs in store.py, Python binds the name 'psycopg2' in store.py's namespace to the psycopg2 module object
- patch('psycopg2.connect'): replaces connect in the psycopg2 MODULE's namespace — but store.py's already-bound reference still points to the original
- patch.object(store_module.psycopg2, 'connect'): replaces connect ON THE OBJECT that store.py holds — the reference store.py uses is now patched
- Concrete example: store.py has 'import psycopg2' at module level; test does 'import src.ingestion.store as store_module'; patches store_module.psycopg2.connect
- Why module-level import matters: if psycopg2 is imported inside a function (def upsert(): import psycopg2), each call re-imports fresh — patch.object on the module reference doesn't work; you'd need to patch the package directly
- The project's bug fix: Bugs #39-41 moved psycopg2, OpenAIEmbeddings, and LangChain loader imports from inside functions to module level — specifically to enable patch.object
- Rule inversion: if you find yourself using patch('psycopg2.connect') and it's not working, the import is almost certainly inside a function — move it to module level

```
# store.py — module-level import (correct) import psycopg2 # at top of file def upsert(chunks): conn = psycopg2.connect(...) # uses module-level reference # test_day19.py — correct patch target import src.ingestion.store as store_module def test_upsert(monkeypatch): mock_conn = MagicMock() # Patch the psycopg2 object that store_module holds monkeypatch.setattr(store_module.psycopg2, 'connect', lambda *a, **kw: mock_conn) store_module.upsert(sample_chunks) mock_conn.cursor.assert_called()
```

Q05 Intermediate

How do you test InformationAgent in isolation? Walk through the injection pattern — what fixture creates the mock service and how does the agent receive it?

Hint: The injection pattern is consistent across all five agents — explain it precisely.

Key points to cover:

- conftest.py fixture: mock_data_service() creates MockDatabricksService() — returns canned AgentResult with realistic metrics data
- test_information_agent.py: @pytest.fixture def agent(mock_data_service) — constructs InformationAgent(service=mock_data_service)
- No patch() needed: InformationAgent receives IDataService via constructor — test passes mock directly; agent never imports DatabricksService
- Test body: result = agent.execute(AgentRequest(query='show retention metrics')) — calls the real execute() method with the mock service underneath
- Assertions: assert result.success is True; assert 'retention' in result.data; assert len(result.sources) > 0
- Why no patch: patch() is a last resort for code you can't refactor; dependency injection is cleaner — the mock is explicit, not hidden in a decorator
- Protocol conformance: conftest.py also asserts isinstance(mock_data_service, IDataService) — ensures MockDatabricksService satisfies the runtime_checkable Protocol
- Same pattern for all five agents: InformationAgent(IDataService), KnowledgeAgent(IVectorService), MetadataAgent(IMetadataService), CapacityAgent(ITicketService), RuleAgent(DB connection mock)

```
# conftest.py @pytest.fixture def mock_data_service(): return MockDatabricksService() # canned responses, no network @pytest.fixture def information_agent(mock_data_service): return InformationAgent(service=mock_data_service) # test_information_agent.py def test_execute_success(information_agent): request = AgentRequest( query='show retention metrics for Q4', thread_id='test-thread', data_products=['retention'] ) result = information_agent.execute(request) assert result.success is True assert result.data != {} assert result.sources != []
```

Q06 Advanced

Your FastAPI tests in test_day20_production.py patch _run_graph instead of get_graph. Why this specific patch target and what was Bug #43?

Hint: Two levels of indirection — understand which one to patch and why.

Key points to cover:

- Bug #43: original test patched get_graph() but the FastAPI endpoint called _run_graph() which called get_graph() internally — patching get_graph didn't stop the real graph from running
- _run_graph() is the actual execution point — it calls get_graph().ainvoke(); patching _run_graph() intercepts BEFORE the graph is invoked at all
- patch('src.api.app._run_graph'): replaces _run_graph in app.py's namespace with a mock that returns a pre-built result dict
- Why not patch get_graph: get_graph is called inside _run_graph; even if get_graph returns a mock graph, ainvoke() on the mock graph must be configured — more setup for the same outcome
- Mock return value: mock_run_graph.return_value = {'final_summary': 'Test summary', 'confidence': 0.9, 'sources': [], 'errors': [], 'execution_ms': 100}
- TestClient: FastAPI TestClient from starlette.testclient — runs the app in a test context, no real uvicorn server needed
- Test flow: patch _run_graph → POST /query with TestClient → endpoint calls mocked _run_graph → assert response JSON matches expected shape
- Lesson: patch the outermost function you control in the call chain; minimise the amount of real code that runs in the test

```
# test_day20_production.py from unittest.mock import patch, AsyncMock from starlette.testclient import TestClient from src.api.app import app MOCK_RESULT = { 'final_summary': 'Test summary', 'confidence': 0.9, 'sources': [], 'errors': [], 'execution_ms': 150, } def test_query_endpoint(): with patch('src.api.app._run_graph', new=AsyncMock(return_value=MOCK_RESULT)): client = TestClient(app) resp = client.post('/query', json={'query': 'test', 'thread_id': 'abc'}) assert resp.status_code == 200 assert resp.json()['final_summary'] == 'Test summary'
```

Smoke Tests, Fixtures & fakeredis

Q07 Intermediate

Explain the 15 smoke tests in test_day20_smoke.py. What do they test and how do they run without any real external services?

Hint: Smoke tests are the widest tests in the suite — explain their scope and isolation.

Key points to cover:

- Smoke tests invoke the full LangGraph graph (get_graph().invoke()) with real nodes, real routing, real guardrails — but all external calls mocked
- ENABLE_MOCK=true: factory.py returns MockDatabricksService, NullVectorService, MockCollibraService, MockJiraService — all agents execute but return canned data
- _MockLLM: no GROQ_API_KEY in test environment → get_llm() returns _MockLLM → supervisor and synthesizer run with stub LLM, return fixed strings
- REDIS_ENABLED=false: cache.py skips Redis; @cached_node falls through to execute the node every time — no fakeredis needed for smoke tests
- MemorySaver: no DATABASE_URL → checkpointer returns MemorySaver — in-memory; sufficient for single-test-run conversation history
- 15 scenarios covered: full_diagnostic intent, data_quality intent, write_rule, HITL two-pass, guardrail block (SQL injection), guardrail PII redaction, unknown intent fallback, empty query, long query, multi data_product, confidence threshold, each agent path independently
- What smoke tests catch: routing bugs (wrong agent selected), state field omissions (_agents_ran missing), synthesizer crash on empty agent_results, guardrail bypass
- Runtime: ~2-5 seconds total — no network, no LLM API calls; fast enough for every CI run

---

Q08 Intermediate

When and how do you use fakeredis in tests? What does it replace and what real Redis behaviour does it NOT simulate?

Hint: fakeredis is used for cache and rate limiter tests — explain the scope and gaps.

Key points to cover:

- fakeredis: an in-process Python implementation of Redis — same redis-py API, no network connection needed
- Used in: test_day14.py (Redis cache tests) — replaces the real Redis client in cache.py with a fakeredis.FakeRedis() instance
- Setup: monkeypatch.setattr(cache_module, '_redis_client', fakeredis.FakeRedis()) — replaces the module-level Redis client
- What it simulates: GET/SET/DELETE with TTL, SCAN cursor, pipeline, SET NX EX (nonce deduplication) — sufficient for testing @cached_node and invalidate_pattern
- What it does NOT simulate: Redis Cluster mode (no cross-slot operations), Redis Pub/Sub latency, TLS/AUTH behaviour, eviction under memory pressure
- What it does NOT simulate: persistence — fakeredis resets between tests (by default); real Redis persists across test runs if not flushed
- Test isolation: each test gets a fresh FakeRedis() instance — no shared state between tests (unlike a real Redis that must be flushed between tests)
- Rate limiter tests: slowapi Limiter with memory:// storage is easier to test than Redis storage — fakeredis used for cache tests; memory:// used for rate limiter tests

```
# test_day14.py — fakeredis setup import fakeredis import src.core.cache as cache_module
@pytest.fixture def fake_redis(monkeypatch): client = fakeredis.FakeRedis()
monkeypatch.setattr(cache_module, '_redis_client', client) return client def
test_cached_node_hit(fake_redis): # First call — miss, executes node result1 =
cached_information_node(sample_state) assert fake_redis.exists(make_key('information',
sample_state)) # Second call — hit, returns cached result2 = cached_information_node(sample_state)
assert result1 == result2
```

Q09 Gotcha

A test passes in isolation but fails when run in a full pytest suite alongside other tests. What are the likely causes and how do you debug and fix test pollution?

Hint: This is a real bug pattern in your project — test ordering and shared state.

Key points to cover:

- Cause 1: shared module-level state — get_config() singleton caches config on first call; if test A changes env vars and test B calls get_config(), test B gets test A's config
- Fix: call reset_config() in a pytest fixture teardown — or use monkeypatch.setenv() which auto-restores after each test
- Cause 2: live Redis connection — if REDIS_ENABLED is True and a test writes to Redis without cleanup, the next test sees stale cache entries
- Fix: REDIS_ENABLED=false in conftest.py (your project does this); or use fakeredis fixture that resets per test
- Cause 3: LangGraph _graph singleton — get_graph() caches the compiled graph; if test A patches a node and doesn't clean up, test B uses the patched graph
- Fix: use with patch(...) context manager — auto-restores the original after the with block; never use patch.start() without patch.stop()
- Cause 4: test ordering dependency — test A creates a rule in the rule registry; test B asserts the registry is empty; tests pass individually, fail together
- Fix: each test should set up and tear down its own state; use a fresh RuleAgent with empty registry per test via fixture
- Debug: pytest --randomly --randomly-seed=last to replay the failing order; pytest -x --tb=long to see first failure in full

---

Design Depth & Testing Philosophy

Q10 Advanced

Your project has unit tests, smoke tests, and integration tests. What is missing from this test pyramid and how would you add it?

Hint: Three tiers is good — what would make the strategy complete?

Key points to cover:

- Missing tier 1: contract tests — verify that MockDatabricksService returns data in exactly the shape that real DatabricksService returns; currently this is only asserted via Protocol conformance, not data shape
- Missing tier 2: performance/load tests — no tests verify that the system handles 50 concurrent requests within SLA; add with Locust or pytest-benchmark
- Missing tier 3: property-based tests (Hypothesis) — generate random valid queries and assert system invariants (e.g. guardrail_passed field always set, agent_results never None)
- Missing tier 4: mutation tests (mutmut) — verify your tests actually catch bugs by injecting mutations; 84% line coverage doesn't mean the assertions are strong
- Missing tier 5: golden file tests for synthesizer — capture expected final_summary outputs for standard queries; alert when LLM output shape changes
- Missing tier 6: chaos tests — simulate Redis down, Neon unreachable, LLM timeout — verify fallback chains activate correctly under failure conditions
- Priority order: contract tests (highest value, easiest to add), then load tests (production confidence), then property tests (correctness guarantees)
- Your current suite is strong for a solo project at this stage — these additions are for a production team, not a critique of the current work

---

Q11 Advanced

An interviewer asks: 'You have 240 tests passing and 84% coverage — how confident are you that the system is production-ready?' How do you answer honestly?

Hint: This is a self-awareness question — balance confidence with honesty about gaps.

Key points to cover:

- Confident: core business logic is well-tested — all five agents, intent routing, guardrails, cache, retry, HITL two-pass pattern all have direct test coverage
- Confident: no regressions — the day-numbered test files ensure every feature added on a given day still works; incremental build with continuous testing
- Confident: the mock/real switching is validated — test_services.py confirms both mock and real services satisfy the Protocol; factory switching is tested
- Less confident: real external service integration — DatabricksService, CollibraService, JiraService are tested only via mocks; real API behaviour (rate limits, auth token expiry, schema changes) is untested
- Less confident: concurrent load — all tests are single-threaded; concurrent HITL approvals, cache stampede, and rate limit enforcement under load are not tested
- Gap: Streamlit UI — no UI tests; a broken _render_hitl_panel() would only be caught by manual testing
- Production readiness requires: passing integration test suite against staging, load test results within SLA, security penetration test on guardrails and HMAC, and runbook for common failure scenarios
- Honest answer: the code is production-quality; the test suite gives high confidence in logic correctness; integration and load testing are the remaining gaps before full production confidence

---

Q12 Design

An interviewer asks: 'Why did you build tests day by day as you added features rather than writing them all at the end? What is the practical difference?' Defend your approach.

Hint: This is a testing philosophy question — TDD-adjacent incremental testing.

Key points to cover:

- Writing tests at the end: you test the system you built; tests often conform to the implementation rather than specifying desired behaviour; harder to isolate bugs in a large codebase
- Incremental testing (your approach): each day's feature is tested before the next day's feature is built on top of it — regressions are caught immediately when they occur
- Practical difference: on Day 15 (retry + HITL), if a test fails, the bug is in Day 15's code — not buried in 20 days of accumulated changes
- Mocking discipline forced by incremental testing: you MUST design for testability early — if InformationAgent had a hardcoded DatabricksService, Day 6 tests would require a live warehouse; injection was chosen for testability
- Bug #38 example: get_graph imported inside function broke tests on Day 20 — caught immediately because tests existed from Day 1; moved to module level as part of that day's bug fix cycle
- Coverage as a forcing function: fail_under=80 means you cannot add untested code and ship — every feature must be testable enough to reach 80%; drives design toward testable code
- Portfolio benefit: 240 tests show not just that you can write code, but that you understand quality engineering — interviewers value this more than raw feature count
- Drawback: day-numbered test files accumulate technical debt over time — ideally refactored into domain-based test modules (test_graph.py, test_agents.py) after initial build

---